

LLD = Low-Level Design.

It's the phase of software design that comes after High-Level Design (HLD), and it focuses on *how* individual components/classes are actually built, as opposed to *what* the overall system architecture looks like.

HLD vs LLD — the distinction

	High-Level Design (HLD)
Focus	Overall system architecture
Answers	"What are the major pieces and how do they talk to each other?"
Typical content	Services, databases, load balancers, APIs, data flow between systems, scalability strategy
Example question	"Should Uber's ride-matching be a separate microservice from payments?"
Audience	Architects, senior engineers, sometimes non-engineers

What LLD actually involves

When someone says "practice LLD" (very common in software engineering interviews, especially at companies doing OOP-heavy backends), they usually mean designing things like:

1. **Class diagrams** — what classes exist, their attributes, methods, and relationships (inheritance, composition, association)
2. **Design patterns** — Factory, Strategy, Observer, Singleton, etc. — applied to solve specific problems cleanly
3. **OOP principles (SOLID)** — Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
4. **Interfaces/abstraction** — exactly what you were just practicing with `Car / SportsCar`: defining a contract separate from implementation

5. **Concrete, compilable/runnable code** — LLD isn't just diagrams; it usually results in actual working class implementations

Classic LLD interview/practice problems

- Design a **Parking Lot** system
- Design a **Library Management** system
- Design an **Elevator** system
- Design a **Vending Machine**
- Design **Tic-Tac-Toe** or **Chess**
- Design a **Car Rental** system (your example is basically a slice of this)
- Design **Splitwise** (expense sharing)